LINUX / SYSTEMS

cgroups & Namespaces Visual Guide

The 7 namespaces, cgroup v2 controllers, and how containers use both.

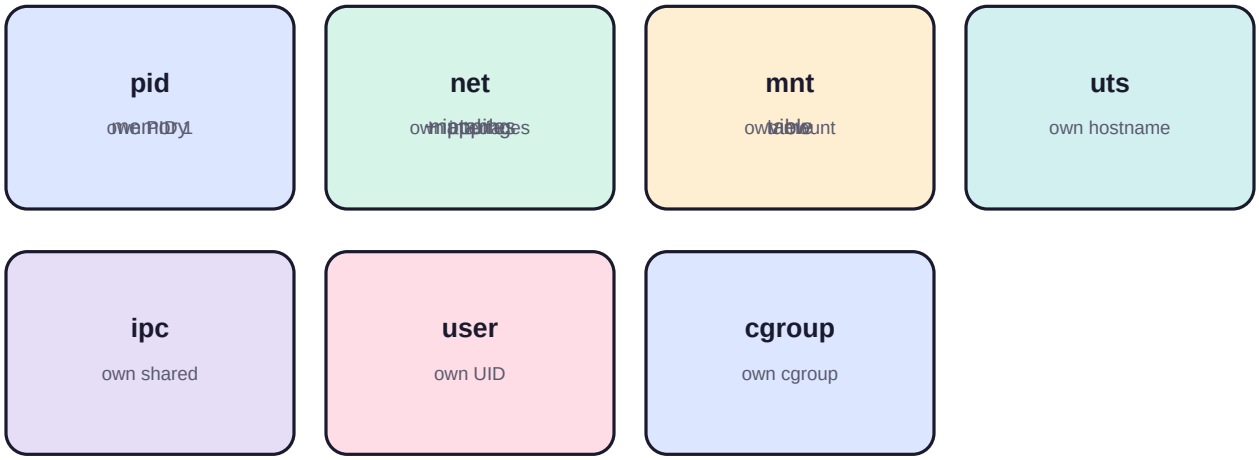
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The 7 Namespaces: What a Process Sees

Namespaces = isolation of VIEW. Each hides a global resource.



A container is just a process with (mostly) all seven namespaces switched on — same kernel, different view of the world.

```
$ lsns                          # every namespace on the system
$ sudo unshare --pid --fork --mount-proc bash   # DIY container-ish shell
```

STEP 2/6

Namespace Demo: unshare

Create the isolation yourself in one command.

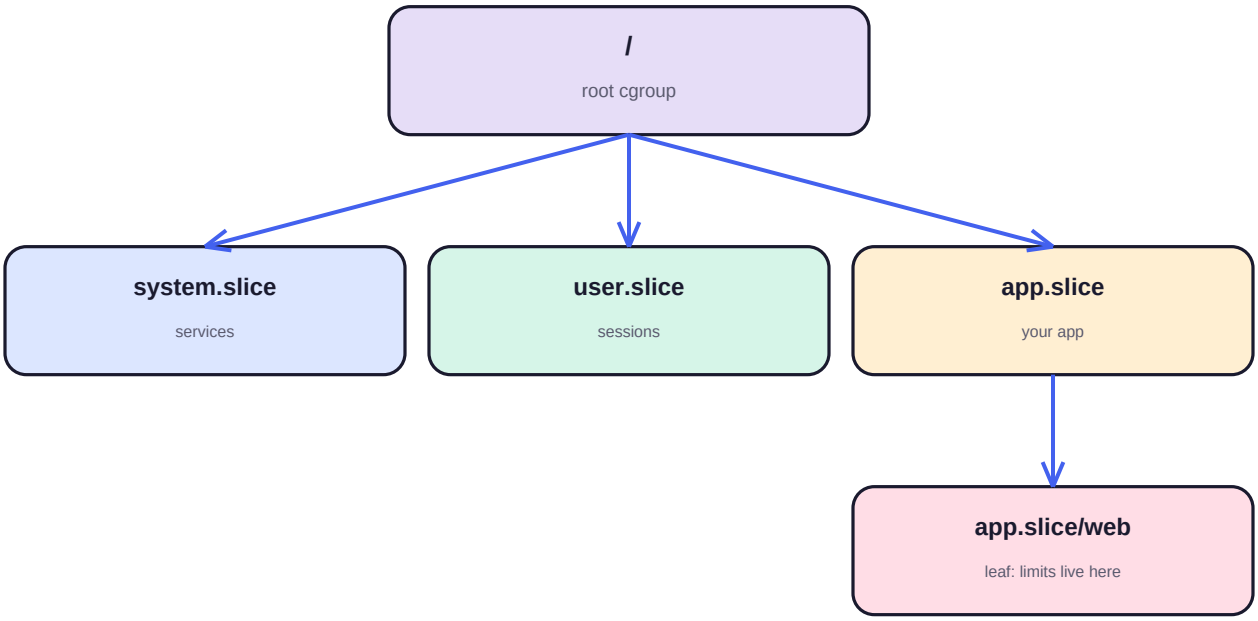
```
$ sudo unshare --pid --fork --mount-proc --net bash
# inside:
# ps aux        # you're PID 1, alone in here
# ip addr       # no interfaces but lo — own net namespace
# hostname c1; hostname   # own UTS, host unaffected
# exit          # namespaces die with the last process
```

- unshare is the teaching tool: it proves namespaces are just kernel flags, not magic.
- Each flag (--pid, --net, --mnt, --uts, --ipc, --user) flips one namespace.
- Container runtimes do exactly this, then add filesystem + cgroup setup around it.

STEP 3/6

cgroup v2: The Resource Hierarchy

cgroups = isolation of RESOURCES. One unified tree at /sys/fs/cgroup.




```
$ systemd-cgls                  # the tree, readable
$ cat /sys/fs/cgroup/app.slice/web/memory.max
$ cat /sys/fs/cgroup/app.slice/web/cpu.weight
```

STEP 4/6

Controllers: The Knobs

Each controller caps one resource dimension.

Controller	File	Does
cpu	cpu.max / cpu.weight	cap or share CPU time
memory	memory.max	hard RAM cap → OOM kill inside group
io	io.max	throttle block device throughput
pids	pids.max	fork-bomb insurance
cpuset	cpuset.cpus	pin to specific cores

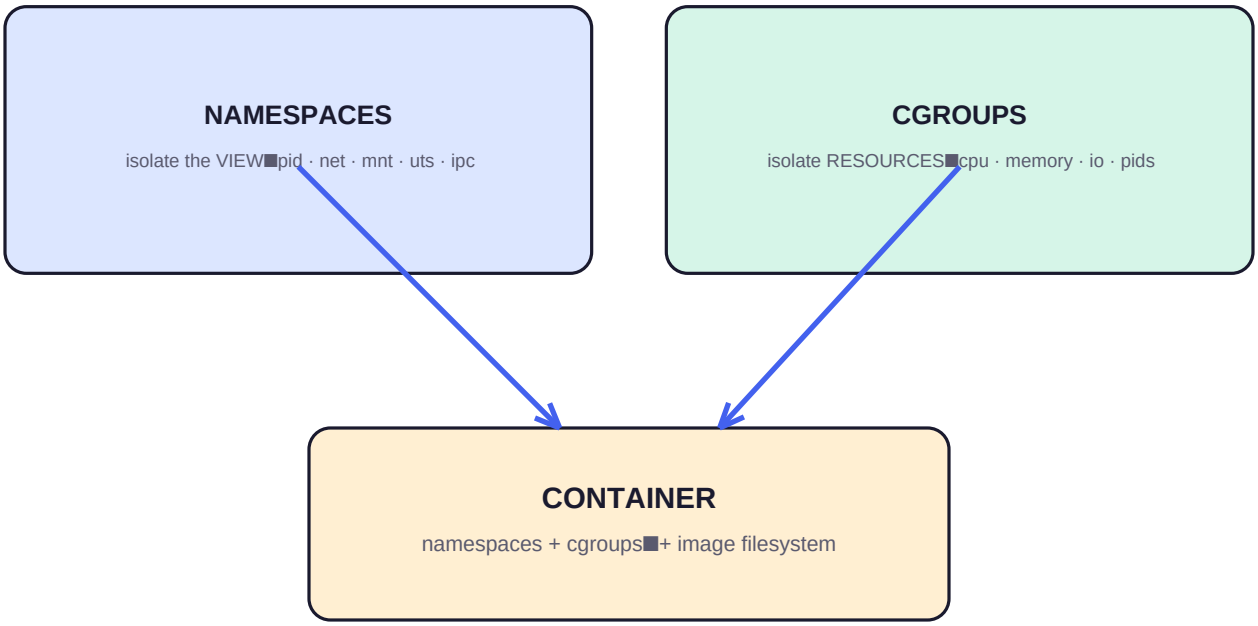
```
$ sudo mkdir /sys/fs/cgroup/demo
$ echo 100M | sudo tee /sys/fs/cgroup/demo/memory.max
$ echo $$ | sudo tee /sys/fs/cgroup/demo/cgroup.procs   # join it
```

memory.max is a HARD wall: exceed it and the kernel OOM-kills inside your group. That's the mechanism behind every container memory limit.

STEP 5/6

Containers = Namespaces + cgroups

Two kernel features, one product.



- docker run --memory=512m --cpus=2 = cgroup limits; --network/--pid = namespaces.
- No virtual hardware, no guest kernel — that's why containers start in milliseconds.
- The price: weaker isolation than VMs (shared kernel). Choose per threat model.

STEP 6/6

systemd Already Uses cgroups

Every service you run is already in the hierarchy.

```
# /etc/systemd/system/api.service
[Service]
MemoryMax=1G
CPUQuota=50%
TasksMax=100

$ systemctl show api.service -p MemoryMax,CPUQuota
$ systemd-cgtop     # top, but per cgroup
```

- You don't need Docker to get resource limits — systemd unit directives ARE cgroup settings.
- MemoryMax/CPUQuota/TasksMax in a unit file: the simplest production guardrail there is.
- systemd-cgtop shows which service is eating the machine, grouped properly.

STEP ✓/✓

Key Takeaways

cgroups & Namespaces

- Namespaces isolate what a process SEES (7 kinds); cgroups limit what it can USE.
- One unified cgroup v2 tree at /sys/fs/cgroup; controllers cap cpu, memory, io, pids.
- A container = all namespaces on + cgroup limits + an image filesystem. No magic.
- systemd puts every service in cgroups already — MemoryMax/CPUQuota are free guardrails.
- Weaker isolation than VMs (shared kernel) — match the tool to the threat model.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev